Voice Orchestrator — Mobile Developer Handoff

Karthi Jeyabalan

2026-04-30

Voice Orchestrator — Mobile Developer Handoff

Everything you need to integrate the Voice Orchestrator API into the SmartHome mobile app. All examples are curl — port them to your platform's HTTP client of choice.

---

1. Credentials

Item	Value
API base URL	https://voiceorchestrator.homeadapt.us/api/v1/voice-auth
VAPI assistant id	1a2904b1-61cf-49da-a804-199d8d39fb9f
VAPI public key (browser/SDK only)	0145170b-662c-4db1-983d-b3bf8aeee1b4
Mobile API key — iOS	sk_ios_48b546d143dae04ec9d2c4396ae9155648e80a5ffbaf3377
Mobile API key — Android	sk_and_a4680b4e40ffe9b5133e8118b7d41cf753fa062688bc05a0
Mobile API key — Web	sk_web_37ceccf66f52a679b34ce1ba53eca44d7f67f2c281d15bc1

> Keys are static for v1. Compile via your CI secret store; never commit to git. Tomorrow they become short-lived JWTs — header shape stays `Authorization: Bearer <token>`, only the token source moves.

---

2. Authentication

Every REST request needs:

```
Authorization: Bearer <your-platform-key>
```

Missing or wrong key → `401 { "error": "...", "code": "UNAUTHORIZED" }`.

Examples below use a shell variable:

```
KEY=sk_ios_48b546d143dae04ec9d2c4396ae9155648e80a5ffbaf3377
BASE=https://voiceorchestrator.homeadapt.us/api/v1/voice-auth
```

---

3. Endpoint Reference

The API has six feature groups:

1. **Voice-auth enrollments** — gate an automation behind a voice phrase challenge
2. **Devices, search & favorites** — discover, search, pin and fire HA items
3. **Scene mappings** — register HA webhook scenes
4. **Automations trigger** — fire a scene/script directly (with voice-gate guard)
5. **Voice enable** — provision a VAPI phone number for a user

Plus the **VAPI SDK** for the spoken challenge UX.

3.1 Voice-Auth Enrollments

User toggles “Voice Protect” on a scene → enrollment row created. After that, direct triggers for that scene are blocked until the user passes a voice challenge.

Create an enrollment

```
curl -s -X POST "$BASE/enrollments" \
  -H "Authorization: Bearer $KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "user_ref":        "scott_mobile",
    "home_id":         "scott_home",
    "automation_name": "Main Lights On",
    "ha_service":      "script",
    "ha_entity":       "main_lights_on"
  }'
```

ha_service ∈ {scene, script, switch, light, lock, cover, media_player, climate, input_boolean, fan}. ha_entity is the **suffix only** — main_lights_on, **not** script.main_lights_on.

Response 201:

```
{
  "id":              "a8100373-7f6c-4e27-9ecf-71d19c1ed4cd",
  "user_ref":        "scott_mobile",
  "home_id":         "scott_home",
  "automation_id":   "main_lights_on",
  "automation_name": "Main Lights On",
  "ha_service":      "script",
  "ha_entity":       "main_lights_on",
  "status":          "ACTIVE",
  "max_attempts":    3,
  "cooldown_seconds": 30
}
```

Idempotent on (user_ref, automation_id).

List enrollments

```
curl -s -H "Authorization: Bearer $KEY" \
  "$BASE/enrollments?user_ref=scott_mobile"
```

Response 200:

```
{
  "count": 1,
  "items": [ { "id": "a8100373-…", "automation_name": "Main Lights On", "status": "ACTIVE" } ]
}
```

Pre-flight check before launching VAPI session

```
curl -s -H "Authorization: Bearer $KEY" \
  "$BASE/check?user_ref=scott_mobile&automation_id=main_lights_on"
```

Response 200:

```
{
  "exists":              true,
  "enrollment_id":       "a8100373-…",
  "status":              "ACTIVE",
  "cooldown_remaining_seconds": 0,
  "attempts_remaining":  3,
  "enrollment_required": false
}
```

If exists: false → user hasn’t enrolled this automation; show enrollment UI. If cooldown_remaining_seconds > 0 → display countdown, block voice session.

Pause / resume / revoke

```
# Pause
curl -s -X PATCH "$BASE/enrollments/<id>/status" \
  -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \
  -d '{"status":"PAUSED"}'

# Resume
curl -s -X PATCH "$BASE/enrollments/<id>/status" \
  -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \
  -d '{"status":"ACTIVE"}'

# Permanently revoke (returns 204)
curl -s -X DELETE "$BASE/enrollments/<id>" -H "Authorization: Bearer $KEY"
```

Recent challenge attempts (audit log)

```
curl -s -H "Authorization: Bearer $KEY" \
  "$BASE/challenges?user_ref=scott_mobile&limit=10"
```

Each entry records `result` (PASS / FAIL / TIMEOUT) and `vapi_call_id` for correlating with VAPI dashboards.

3.2 Devices, Search and Favorites

Per-user pinned HA items — devices (Den Lamp, Yale Lock), scenes (Good Morning), scripts, and HA-configured automations. Drives the home-screen tile grid.

3.2.1 Discover physical devices

```
curl -s -H "Authorization: Bearer $KEY" \
  "$BASE/devices/discover?home_id=scott_home"
```

Returns one row per HA *device* (not per entity). Each row includes `device_id`, `name`, `manufacturer`, `model`, `area`, the resolved `primary_entity_id` (the controllable surface — light/switch/lock/…) and the full list of entities bound to that device.

Response `200`:


```
{
  "home_id": "scott_home",
  "count":   65,
  "items": [
    {
      "device_id":        "ca96f20dd98e20c6d43a791775940295",
      "name":             "Yale Lock",
      "manufacturer":     "Yale",
      "model":            "YRD226 TSDB",
      "area":             "Entry",
      "primary_entity_id": "lock.yale_yrd226_tsdb",
      "primary_domain":   "lock",
      "is_controllable":  true,
      "all_entities":     ["lock.yale_yrd226_tsdb"]
    }
  ]
}
```


Cached server-side for 60s per home. Devices with only sensor/diagnostic entities have `is_controllable: false` and cannot be favorited.

3.2.2 Unified search — devices, scenes, scripts, automations

The single endpoint behind the mobile “Add favorite” picker.

```
curl -s -G "$BASE/items/search" \
  -H "Authorization: Bearer $KEY" \
  --data-urlencode "home_id=scott_home" \
  --data-urlencode "q=bat" \
  --data-urlencode "user_ref=scott_mobile"
```

Query params:

Param	Required	Notes
home_id	yes	which home to search
q	no	case-insensitive substring on name + entity_id + device_id
kind	no	comma-separated filter from device,scene,script,automation,entity
user_ref	no	when set, populates is_favorited + favorite_id per row
limit	no	default 200, max 500

Response 200:


```
{
  "home_id": "scott_home",
  "count": 4,
  "items": [
    {
      "kind": "device",
      "device_id": "6b86cd8c539ad69b193a8ff2acbf3b4e",
      "entity_id": "switch.bat_sign",
      "name": "Bat Sign",
      "domain": "switch",
      "manufacturer": "Belkin",
      "model": "Socket",
      "area": "Downstairs Bat Cave",
      "state": "off",
      "is_favorited": true,
      "favorite_id": "26260ec6-ccfb-4cfb-a84c-55a3b458dc23"
    },
    {
      "kind": "automation",
      "device_id": null,
      "entity_id": "automation.utogglebatsign",
      "name": "uBatsignToggle",
      "domain": "automation",
      "state": "on",
      "is_favorited": false,
      "favorite_id": null
    }
  ]
}
```


When is_favorited: true, favorite_id gives you the UUID needed to toggle off viaDELETE /favorites/{id} — no second roundtrip needed.

3.2.3 Add a favorite — by device OR by entity

Required fields: user_ref, home_id, and **exactly one** of device_id or entity_id. Everything else is optional.

Field	When	Notes
user_ref	always	scopes the favorite to one user
home_id	always	scopes the favorite to one home
device_id	for physical things	server resolves entity + name from HA registry
entity_id	for scenes/scripts/automations	format domain.suffix

Field	When	Notes
friendly_name	optional	for device_id adds: defaults to HA's device name. for entity_id adds: defaults to the entity suffix (slug) — usually worth setting explicitly
position	optional	defaults to next-after-last

Favoriting a device (no friendly_name needed — server pulls it from HA):

```
curl -s -X POST "$BASE/favorites" -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" -d '{"user_ref
```

The server resolves entity_id, primary_entity_id, domain, and friendly_name ("Bat Sign") from the HA device registry. You may still override friendly_name if the user has renamed the device in your app.

Favoriting a scene / script / HA automation(no device behind them — friendly_name recommended so you don't get the slug):

```
curl -s -X POST "$BASE/favorites" \
  -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \
  -d '{
    "user_ref":      "scott_mobile",
    "home_id":       "scott_home",
    "entity_id":     "scene.good_morning",
    "friendly_name": "Good Morning"
  }'
```

kind is inferred from the domain prefix (scene/script/automation/entity).

Important constraints: - Send **exactly one** of device_id or entity_id — both → 400, neither → 400. - Devices with only sensors/diagnostics → 400 NO_CONTROLLABLE_ENTITY. - Duplicate (user_ref, home_id, entity_id) → 400. - **Locks auto-enroll**: when the resolved entity is lock.*, the server creates a STEP_UP voice-auth enrollment in the same transaction. The response includes voice_auth_required: true and voice_auth_enrollment_id.

Response 201 (lock device example):


```
{
  "id":                       "440c4234-4fea-4c1c-8717-425e7afdfdea",
  "user_ref":                    "scott_mobile",
  "home_id":                     "scott_home",
  "entity_id":                 "lock.yale_yrd226_tsdb",
  "friendly_name":               "Yale Lock",
  "domain":                    "lock",
  "kind":                    "device",
  "device_id":                   "ca96f20dd98e20c6d43a791775940295",
  "primary_entity_id":           "lock.yale_yrd226_tsdb",
  "position":                  3,
  "voice_auth_required":         true,
  "voice_auth_enrollment_id":    "9c520ce7-bdb5-4143-8e59-091b0626bdf5",
  "created_at":                  "2026-04-30T20:31:46.989419"
}
```


3.2.4 List favorites

```
curl -s -H "Authorization: Bearer $KEY" \
  "$BASE/favorites?user_ref=scott_mobile&home_id=scott_home"
```

Each row includes voice_auth_required (computed at read time from the enrollments table) so the mobile UI can render the lock badge correctly.

Response:

```
{
  "count": 4,
  "items": [
    { "id": "...", "kind": "entity",     "entity_id": "switch.bat_sign",                "voice_auth_required": false },
    { "id": "...", "kind": "scene",      "entity_id": "scene.good_morning",             "voice_auth_required": false },
    { "id": "...", "kind": "automation", "entity_id": "automation.lights_off_at_night", "voice_auth_required": false },
    { "id": "...", "kind": "device",     "entity_id": "lock.yale_yrd226_tsdb",          "voice_auth_required": true  }
  ]
}
```

Items are ordered by `position`.

3.2.5 Fire a favorite (one-tap activation)

```
curl -s -X POST "$BASE/favorites/<id>/fire" \
  -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \
  -d '{}'
```

The server resolves the HA action automatically per domain: - scenes / scripts / lights / switches → `turn_on` - HA automations → `trigger` - locks → `unlock`

Response `200` on success:

```
{ "success": true, "message": "ok", "status_code": 200, "latency_ms": 142,
  "favorite_id": "...", "entity_id": "switch.bat_sign" }
```

Voice-gate guard — `409 ENROLLMENT_REQUIRED`:

```
{
  "error": "this favorite requires voice authentication",
  "code":  "ENROLLMENT_REQUIRED",
  "enrollment_id":   "9c520ce7-...",
  "automation_name": "Yale Lock",
  "automation_id":   "yale_yrd226_tsdb",
  "home_id":         "scott_home"
}
```

When you see `409`, route the user into the VAPI session (§3.6) using the returned `automation_id` and `home_id` — the lock will be unlocked only after the spoken phrase challenge passes. **This is non-bypassable for locks** — every lock favorite is auto-enrolled.

3.2.6 Remove a favorite

```
curl -s -X DELETE "$BASE/favorites/<id>" -H "Authorization: Bearer $KEY"
```

Returns `204` on success, `404` if id is unknown.

3.2.7 Reorder (drag-and-drop UI)

```
curl -s -X PATCH "$BASE/favorites/reorder" \
  -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \
  -d '{
    "items": [
      {"id": "<id-1>", "position": 0},
      {"id": "<id-2>", "position": 1}
    ]
  }'
```

3.2.8 Worked example — search, add, remove a device

End-to-end round-trip verified on production. Copy-paste, replace `USER` with your test user, and run top-to-bottom:

```
KEY=sk_ios_48b546d143dae04ec9d2c4396ae9155648e80a5ffbaf3377
BASE=https://voiceorchestrator.homeadapt.us/api/v1/voice-auth
USER=demo_user_42
HOME=scott_home
```

Step 1 — search for devices matching "bat". Returns each with its device_id, name, manufacturer, current state, and per-user is_favorited flag.

```
curl -s -G "$BASE/items/search" \
  -H "Authorization: Bearer $KEY" \
  --data-urlencode "home_id=$HOME" \
  --data-urlencode "kind=device" \
  --data-urlencode "q=bat" \
  --data-urlencode "user_ref=$USER"
```

Real response from prod (3 devices):

```
name                 device_id                        state
Bat Cave Echo Show 5    d97e38a608fe463cd3cf0ea4a8e38850     unavailable
Bat Sign             6b86cd8c539ad69b193a8ff2acbf3b4e     off
Bathroom               71525ebcb48461b3bc93d2300994afe6     playing
```

Step 2 — add Bat Sign to favorites by device_id. No need to send entity_id or friendly_name — the server resolves both from the HA registry.

```
curl -s -X POST "$BASE/favorites" -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" -d "{\"user_re
```

Response 201:

```
{
  "id":               "808dc489-eced-4327-bc60-ab1a690997b3",
  "user_ref":           "demo_user_42",
  "home_id":            "scott_home",
  "entity_id":        "switch.bat_sign",
  "friendly_name":      "Bat Sign",
  "domain":           "switch",
  "kind":             "device",
  "device_id":          "6b86cd8c539ad69b193a8ff2acbf3b4e",
  "primary_entity_id":   "switch.bat_sign",
  "position":           0,
  "voice_auth_required": false,
  "created_at":         "2026-04-30T21:56:59.422121"
}
```

Save the returned id (808dc489-...) — it's the favorite UUID you'll use for delete and fire.

Step 3 — remove the favorite by its UUID.

```
curl -s -X DELETE "$BASE/favorites/808dc489-eced-4327-bc60-ab1a690997b3" \
  -H "Authorization: Bearer $KEY" -w "HTTP %{http_code}\n"
```

Response: HTTP 204 (empty body). A subsequent GET /favorites?user_ref=$USER&home_id=$HOME will show count: 0.

Notes for the implementation:

- For a *lock* device (e.g. device_id of the Yale Lock), step 2 returns voice_auth_required: true and includes voice_auth_enrollment_id. Tapping the tile next must hit /favorites/{id}/fire, which returns 409 ENROLLMENT_REQUIRED — route to VAPI from there.
- For a *scene/script/automation*, send entity_id instead of device_id in step 2: {"user_ref":..., "home_id":..., "entity_id":"scene.good_morning"}.
- The mobile app should always re-fetch favorites via GET /favorites?user_ref=...&home_id=... after any add/remove — the server is the single source of truth (no client-side cache).

3.3 Scene Mappings

Map a friendly scene name (e.g. "decorations on") to an HA webhook id. This is what lets Alexa or VAPI translate spoken names into webhook calls.

Create a scene mapping

```
curl -s -X POST "$BASE/scene-mappings" \
  -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \
  -d '{
    "home_id":    "scott_home",
    "scene_name": "Movie Night",
    "webhook_id": "movie_night_1751404299018"
  }'
```

Scene names are normalized to lowercase server-side.

Response 201:

```
{
  "id":         "3e849ff7-be41-4bd2-aae4-eb139f3a0c9b",
  "home_id":    "scott_home",
  "scene_name": "movie night",
  "webhook_id": "movie_night_1751404299018",
  "is_active":  true,
  "created_at": "2026-04-28T15:30:09.818419"
}
```

List scenes for a home

```
curl -s -H "Authorization: Bearer $KEY" \
  "$BASE/scene-mappings?home_id=scott_home"
```

Update a scene (rename, swap webhook, or deactivate)

```
curl -s -X PATCH "$BASE/scene-mappings/<id>" \
  -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \
  -d '{"webhook_id": "new_webhook_id_here"}'
```

Body fields are all optional: scene_name, webhook_id, is_active.

Delete

```
curl -s -X DELETE "$BASE/scene-mappings/<id>" -H "Authorization: Bearer $KEY"
```

3.4 Automation Trigger

Direct fire-and-forget activation. Used when the user taps a favorite tile or a “scenes” button. **Refuses with 409 if a voice-auth enrollment gates the automation** — the app must then route through the VAPI session instead.

Fire an automation

```
curl -s -X POST "$BASE/automations/trigger" \
  -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \
  -d '{
    "home_id":      "scott_home",
    "ha_service":   "scene",
    "ha_entity":    "movie_night",
    "user_ref":     "scott_mobile",
    "automation_id": "movie_night"
  }'
```

user_ref and automation_id are **required** for the voice-gate check — omit them and the gate is bypassed (only do that for clearly non-protected items).

Response 200:

```
{ "success": true, "message": "OK", "status_code": 200, "latency_ms": 142 }
```

Response 409 (voice-auth enrollment exists for this automation):

```
{
  "error":       "this automation requires voice authentication",
  "code":         "ENROLLMENT_REQUIRED",
  "enrollment_id": "a8100373-…"
}
```

When you see `409 ENROLLMENT_REQUIRED`, start the VAPI SDK with the appropriate `variableValues` (see §3.6) instead of retrying this endpoint.

Response `502` if Home Assistant is unreachable; `400` for validation errors.

3.5 Voice Enable (VAPI provisioning)

Provision a dedicated phone number for the user via VAPI. After this, the user can place outbound calls TO that number to authenticate by voice.

Enable voice for a user

```
curl -s -X POST "$BASE/voice-enable" \
  -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \
  -d '{
    "user_ref":  "scott_mobile",
    "home_id":   "scott_home",
    "area_code": "415"
  }'
```

`area_code` is optional; if omitted, VAPI picks one.

Response `201`:

```
{
  "id":                  "432da968-9f19-4fba-b92c-36bee112c7f1",
  "user_ref":            "scott_mobile",
  "home_id":             "scott_home",
  "phone_e164":          "+14151702028",
  "vapi_phone_number_id": "vpn_abc123…",
  "is_active":           true
}
```

Idempotent — calling again for the same `(user_ref, home_id)` returns the existing mapping without a second VAPI purchase. **Billable on the VAPI side** when first called.

Status check

```
curl -s -H "Authorization: Bearer $KEY" \
  "$BASE/voice-enable?user_ref=scott_mobile"
```

Response `200`:

```
{
  "enabled":    true,
  "is_dry_run": false,
  "mapping": { "phone_e164": "+14151702028", "vapi_phone_number_id": "vpn_abc123…" }
}
```

`is_dry_run: true` means the server hasn't been configured with a live VAPI key — useful for staging environments. Production should always show `false`.

Disable (release the number)

```
curl -s -X DELETE "$BASE/voice-enable?user_ref=scott_mobile" \
  -H "Authorization: Bearer $KEY"
```

Returns `204` on success, `404` if no active mapping.

3.6 VAPI SDK — Voice Challenge

When the user taps a voice-protected automation (or after a 409 ENROLLMENT_REQUIRED from /automations/trigger), launch the VAPI SDK with the public key and these variableValues:

```
assistantId: "1a2904b1-61cf-49da-a804-199d8d39fb9f"

assistantOverrides.variableValues:
  home_id:         "scott_home"
  user_ref:        "scott_mobile"
  automation_id:   "main_lights_on"
  automation_name: "Main Lights On"
  initiated_by:    "MOBILE_IOS" | "MOBILE_ANDROID"
```

The assistant says *"Confirm Main Lights On? Say the security phrase."* — the user repeats — Home Assistant fires. Subscribe to VAPI's call-end event to know when it's done; then refresh enrollment state with GET /check.

VAPI SDK references: - iOS: https://github.com/VapiAI/ios - Android: https://github.com/VapiAI/android - Web: https://github.com/VapiAI/web

The SDK uses the **VAPI public key**, not the mobile API key.

---

4. Error Reference

HTTP	Code	Meaning	App action
200 / 201 / 204	—	OK	proceed
400	VALIDATION	Bad request body	fix input, show inline error
401	UNAUTHORIZED	Missing / wrong API key	auth-config bug — do not retry blindly
404	—	Not found	varies (prompt to create, etc.)
409	ENROLLMENT_REQUIRED	Automation/favorite is voice-gated (always for locks)	launch VAPI session instead
400	NO_CONTROLLABLE_ENTITY	Device has only sensors/diagnostics; cannot be favorited	exclude from picker
409	CONFLICT	State conflict (e.g. un-revoke)	surface message
502	VAPI_ERROR	VAPI provisioning failed	retry once with backoff
502	—	Home Assistant unreachable	retry once with backoff
503	NOT_CONFIGURED	Server feature not wired up	report to backend team

Error envelope is always:

```
{ "error": "<human message>", "code": "<OPTIONAL_CODE>" }
```

---

5. Acceptance Checklist

Run through these to verify the integration end-to-end:

- [ ] GET /enrollments?user_ref=demo_user with no Authorization → 401
- [ ] Same call with valid bearer → 200 { count: 0, items: [] }
- [ ] POST /enrollments → 201 with automation_id echoed back
- [ ] GET /check?user_ref=…&automation_id=… → exists: true, attempts_remaining: 3
- [ ] GET /automations/discover?home_id=… → list of HA candidates
- [ ] GET /devices/discover?home_id=… → list of physical HA devices

- [ ] GET /items/search?home_id=…&q=bat&user_ref=… → mixed devices/scenes/automations with is_favorited flag
- [ ] POST /favorites { device_id } → 201, server resolves primary entity
- [ ] POST /favorites { entity_id: "scene.good_morning" } → 201, kind=scene
- [ ] POST /favorites for a sensor-only device → 400 NO_CONTROLLABLE_ENTITY
- [ ] POST /favorites for a lock.* → 201 with voice_auth_required: true and voice_auth_enrollment_id populated
- [ ] POST /favorites/{lock_fav_id}/fire → 409 ENROLLMENT_REQUIRED (never 200)
- [ ] GET /favorites → ordered list, lock row has voice_auth_required: true
- [ ] POST /automations/trigger for an **un-enrolled** automation → 200 success: true
- [ ] POST /automations/trigger for an **enrolled** automation → 409 ENROLLMENT_REQUIRED
- [ ] Launch VAPI SDK after a 409 → assistant prompts for security phrase → HA fires
- [ ] POST /voice-enable → 201 with phone_e164; second call same payload → idempotent same id
- [ ] DELETE /voice-enable?user_ref=… → 204; status flips to enabled: false

---

6. Contact

When something misbehaves, send the backend team: - The user_ref you used - A vapi_call_id from GET /challenges?user_ref=<you> if it was a voice flow - The HTTP request + response (curl -i output is ideal)

All tool-call payloads are server-logged with redacted bodies — we can trace any specific call by id.